Algorithmica (1996) 16: 151–160

Algorithmica
© 1996 Springer-Verlag New York Inc.

# Fast Stable In-Place Sorting with $O(n)$ Data Moves[1]

J. I. Munro[2] and V. Raman[3]

**Abstract.** Until recently, it was not known whether it was possible to sort *stably* (i.e., keeping equal elements in their initial order) an array of $n$ elements using only $O(n)$ data moves and $O(1)$ extra space. In [13] an algorithm was given to perform this task in $O(n^2)$ comparisons in the worst case. Here, we develop a new algorithm for the problem that performs only $O(n^{1+\varepsilon})$ comparisons ($0 < \varepsilon \leq 1$ is any fixed constant) in the worst case. This bound on the number of comparisons matches (asymptotically) the best known bound for the same problem with the stability constraint dropped.

**Key Words.** Sorting, Stability, Data moves, In-place.

**1. Introduction and Motivation.** Sorting has been a focus of intense research for more than three decades due to its theoretical and practical significance. Knuth [5] has suggested that it can be viewed as a paradigm for most computing problems. Initial research on sorting centered on devising fast algorithms and, in particular, minimizing the number of comparisons needed. Once optimal $O(n \lg n)$ algorithms were available, research shifted to other issues such as *in-place* sorting, i.e., using $O(1)$ indices and extra data locations [16], [15], [3] [14], *stable* sorting, i.e., keeping equal elements in their initial order [15], [3], sorting multisets optimally [10], [4], and more recently to sorting in as few data moves as possible [11]. In a recent paper [13] we confronted all of these issues simultaneously, by designing the first *stable in-place* sort that performs $O(n)$ data moves. The method requires $O(n^2)$ comparisons in the worst case. In this paper we improve this bound to $O(n^{1+\varepsilon})$ where $\varepsilon \in (0, 1)$ is any fixed constant.

In-place algorithms have a particular advantage in external sorting environments. They help increase the amount of input data that can be brought to the main memory in one I/O operation, thereby reducing the number of I/O operations. Stability is a required feature when sorting a list with respect to several different keys. Furthermore, when each element of the list to be sorted is large, data move becomes a significant operation. Thus a stable in-place sort with $O(n)$ data moves would also be of practical interest.

The difficulty of designing stable in-place algorithms can be seen from the complexity, increase in the constant factor, and the history of:

1. The first stable in-place $O(n)$ merging algorithm [15] (1977) over the first unstable in-place $O(n)$ merging algorithm [6] (1969), [5, Exercise 5.2.4. Problem 18].

---

[1] A version of this paper appeared in the *Proceedings of the 11th FST & TCS Conference* [9]. This research was supported by NSERC of Canada grant No. A-8237 and the ITRC of Ontario.

[2] Department of Computer Science, University of Waterloo, Waterloo, Ontario, Canada N2L 3G1. imunro@uwaterloo.ca.

[3] The Institute of Mathematical Sciences, C.I.T. Campus, Madras, India 600113. vraman@imsc.ernet.in.

Received August 5, 1993; revised October 19, 1994. Communicated by R. Sedgewick.

2. The first stable in-place $O(n \lg n)$ sorting algorithm [15] (1977) over the first unstable in-place $O(n \lg n)$ sort, Heapsort [16] (1964).
3. The first stable in-place algorithm to sort a multiset optimally [4] (1992) over the first unstable in-place algorithm that sorts a multiset optimally [10] (1991).
4. The first stable in-place sorting algorithm performing $O(n)$ data moves [13] (1990) over the first unstable in-place $O(n)$ moves sort, Selection Sort [5], [1] (1956).

Furthermore, for the first three problems, the asymptotic complexity has been established to be the same for both the unstable and stable versions. In fact, once these problems were well understood, the complexity and the constant factor have been improved significantly [2], [3] for these (at least for the first two) problems.

However, we are behind on the problem of sorting in-place with $O(n)$ data moves. The best known unstable algorithm [11] for the problem uses $O(n^{1+\varepsilon})$ comparisons in the worst case. (The $\varepsilon$ is an artifact of the requirement of a linear number of data moves; its reduction results in an increase in the number of data moves.) For the stable version of the problem, until recently, it was not even known whether there is a stable in-place sort performing $O(n)$ data moves regardless of the number of comparisons. In this paper we improve the previously known $O(n^2)$ bound to $O(n^{1+\varepsilon})$ for the stable version. This proves that the asymptotic complexity for the stable version is the same as the known upper bound for the unstable version for the fourth problem as well. To achieve our stable in-place $O(n^{1+\varepsilon})$ sort, we use some of the results and space-saving techniques developed recently [13], [11], [14], [10]. It is still unknown whether there is an in-place sort (stable or unstable) that performs $O(n)$ data moves and $O(n \lg n)$ comparisons in the worst case.

As our primary objective in this paper is to prove the existence of an $o(n^2)$ algorithm satisfying the constraints on the space, stability, and data moves, we do not concern ourselves with performing certain steps most efficiently. In most computing problems the discovery of the asymptotically fast algorithm precedes that of the practical and efficient one. While we feel that the development of a practical method is important, we are simply not that far along in our understanding of sorting with these constraints.

In the next section we present a stable $O(n^{1+\varepsilon})$ sort that uses $O(n)$ data moves and about $2n+o(n)$ bits of extra storage. The following section presents a technique to encode bits into the input data without modifying the data values. Section 4 describes how the bits used in the algorithm of Section 2 can be encoded using the encoding technique, and presents a complete analysis of the general algorithm. Section 5 concludes with a discussion of our result and the techniques.

Throughout this paper, $0 < \varepsilon < 1$ is any fixed constant. All logarithms in this paper are to the base 2 and are denoted by lg; and $\lg^*$ denotes the number of times the logarithm may be taken before the quantity becomes at most zero. We follow the convention that elements are to be sorted into nondecreasing order and are given in an array. We generally speak of the array as running from left to right. To facilitate the discussion, we ignore the notations to round up or round down noninteger values. This does not affect the asymptotic analysis of the algorithms.

By the *rank* of an element, we mean the position in the array the element should occupy after the final stable sort of the array is completed.

**2. Stably Sorting with $O(n)$ Movements Using $2n$ Bits.** In [13] we first developed a simple $O(n^2)$ stable sort that performs $O(n)$ data moves using $n$ bits (tags) of extra memory. Our initial observation here is that if we have another about $n$ bits, by judicially using them, we can design an $O(n^{1+\varepsilon})$ algorithm to prove the following theorem.

THEOREM 1. *Let $L$ be an array of $n$ items, and let $0 < \varepsilon < 1$ be any fixed constant. Then $L$ can be stably sorted in $O(n^{1+\varepsilon})$ comparisons in the worst case, using $O(n)$ data moves and $2n + o(n)$ bits of extra memory plus a constant number of indices and data locations.*

PROOF. In the following description, let $\delta < \varepsilon/2$ be any fixed positive constant. (The reason for this restriction on $\delta$ will be clear in Section 4. For the proof of this theorem, a $\delta < \varepsilon$ is sufficient.)

The algorithm we develop has the following three phases; the details of each phase are described later.

1. Find the elements of ranks $in^{1-\delta}$, for $i = 1$ to $n^{\delta}$, and place them in their correct locations stably with respect to the other elements.

   These special elements are called *guides* and the set of locations between any two consecutive guides is called a *block*. Thus we have $n^{\delta}$ blocks. This phase uses $n$ bits to identify the guides, and another $n^{\delta} \lg n$ bits to store their initial positions.
2. Stably partition all other elements into their respective blocks. That is, at the end of this phase, all elements in a block (though unsorted) belong to that block in the final sorted list.

   This phase uses $n$ bits as tags to distinguish elements that are moved to their final block from the rest. It uses another $n$ bits as approximately $n/\lg n$ counters.
3. Sort each block stably.

   This step is achieved by recursing till the number of elements in each block reaches a manageable level, and then by applying our stable in-place $O(n^2)$ sort that uses $O(n)$ data moves [13].

At the end of the third phase, the list is stably sorted. We elaborate on each phase by describing the specific details.

*Phase* 1. There are two issues of concern here: to find the prescribed ranked elements and to place them in their locations stably.

- *Step* 1.1: *Finding the guides.* In a recent paper [12] we describe an in-place algorithm to select the $k$th smallest element, for any $k$, using $O(n^{1+\gamma})$ comparisons (where $0 < \gamma < 1$ is any constant) from a list of $n$ distinct elements residing in a read-only memory (i.e., without performing *any* data move). As equal-valued elements have their own relative order among themselves (depending on their initial positions), we can repeatedly apply the selection algorithm to find the elements of ranks $in^{1-\delta}$, for $i = 1$ to $n^{\delta}$, using no data move. We use $n$ bits to denote whether or not an element is of the rank of interest. The bits are initialized to 0 for all elements. Once we find an element of desired rank, we set the bit corresponding to its position to 1. As we see later, its

actual rank can be retrieved later. The guides are also called tagged elements as their bits are set to 1 at the end of this phase.

The initial positions of the guides are required in the next phase to resolve ties between the guides and the nonguides of the same value. Since the guides are moved in the next step, we store in another $n^\delta$ counters $g_i$, for $i = 1$ to $n^\delta$, each taking $\lg n$ bits, the initial positions of elements of ranks $in^{1-\delta}$, for $i = 1$ to $n^\delta$, in that order. This information is used in Phase 2 to partition all elements stably with respect to the guides. Thus, once we find an element of the desired rank, its tag bit is set to 1, and its (initial) position is stored in the corresponding counter.

Choosing $\gamma$ to be $\varepsilon - \delta$, this step can be accomplished using $O(n^{1+\delta+\gamma}) = O(n^{1+\varepsilon})$ comparisons and $n + O(n^\delta \lg n)$ bits.

- *Step* 1.2: *Placing the guides in their destinations.* At the beginning of this step, the element whose destination is $in^{1-\delta}$ is the (tagged) element in position stored in the counter $g_i$. Therefore, even with the information retained from Step 1.1, it requires reading all the $n^\delta g_i$ counters to determine the destination of a given tagged element. However, if we stably separate and sort the tagged elements first, it is easier to determine the destination of each tagged element. Separating stably the tagged elements from the others is equivalent to sorting stably a list consisting of two distinct keys (the tagged and the untagged ones), the first one appearing $n^\delta$ times. It can be achieved using $O(n)$ comparisons and data moves by applying the first phase of the stable in-place two key sort [13]. Here, of course, every time a tagged element is moved, the corresponding tags should also be reset. Once the two sets of elements are separated, the tagged elements can be sorted easily in linear time as there are only $n^\delta$ of them.

Now, we have a sorted list of size approximately $n^\delta$ followed by an unsorted list, and we have the the extra information that the $i$th element of the first list goes to $(in^{1-\delta})$th location of the entire list. Then, as shown in Figure 1, a series of list exchanges of lists of size about $n^{1-\delta}$ can now place the tagged elements in their proper destinations. (By a list exchange, we mean the list $UV$ exchanged to $VU$.)

In the figure the numbers given are the *positions* each element should be moved to at the end of this step. For example, the numbers 4, 8, 12, and 16 are the destinations of the four guides, and the objective is to place them in their destinations stably in-place. First, the last guide and the last three nonguides are exchanged so that the last three elements of the entire list are placed properly. Then the third guide and the element in the position previously occupied by the last guide are exchanged with the previous three nonguide elements. This process is continued until the guides are placed in their destinations. It can be seen that this step involves about $O(n^\delta)$ list exchanges of

4 8 12 16 1 2 3 5 6 7 9 10 11 13 14 15
4 8 12 16 1 2 3 5 6 7 9 10 11 13 14 15
4 8 12 13 1 2 3 5 6 7 9 10 11 14 15 16
4 8 9 10 1 2 3 5 6 7 11 12 13 14 15 16
4 5 6 7 1 2 3 8 9 10 11 12 13 14 15 16
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

**Fig. 1.** Placing the guides in their destination.

lists of size about $n^{1-\delta}$ and hence requires $O(n)$ data moves (and no comparisons) in total.

Thus, this step performs $O(n)$ comparisons and $O(n)$ data moves using the $n$ bits.

*Phase* 2. In this phase we regard the value of all elements that belong (in the final sorted list) to a particular block to be the same. So, stably partitioning all elements into their corresponding blocks can be thought of as stably sorting a list consisting of $n^{\delta}$ distinct keys, each appearing approximately $n^{1-\delta}$ times.

Suppose that every nonguide element is in the position to which it was moved to at the end of Phase 1. We can find the final block of a nonguide element by doing a binary search on the guides. If the element ties with a guide, then the final block of the element is to the left or to the right of the guide. From the current position of the element, its initial position can be computed easily by figuring out the number of guides that are moved to and from the locations to its left. More precisely, the initial position of the element is equal to its current position *minus* the number of guides that are moved from locations to its right to locations to its left *plus* the number of guides that are moved from locations to its left to locations to its right. The information about the guides' positions can be found by looking at each of the counters $g_i$'s. Based on the initial position of the element and that of the guide, the tie is resolved.

We perform stable partitioning in two steps as follows. To place an element stably in its block, we require two quantities:

(a) The block corresponding to that element.
(b) The position inside the block it should be placed.

Given an element $L[i] = x$, its proper block $b_x$ can be easily found (using $O(\lg n)$ comparisons) by a binary search on the guides, and using the counters $g_i$, $i = 1$ to $n^{\delta}$, to resolve ties. To determine its proper position inside the block, we have to know the number of elements (excluding the guides) to the left of position $i$ in the initial unsorted list that belong to block $b_x$. For this, we first identify those elements that are moved (to their final destinations) from the rest using $n$ bits. The bits are all initialized to 0, and every time an element is moved to its destination, the bit corresponding to that position is set to 1. Still, computing an element's final position inside its block may require a $\Theta(n)$ scan for each element, thus requiring $\Theta(n^2)$ comparisons in total. To overcome this, we first compute and store sufficient information (using another $n$ bits) in Step 2.1 so that the quantity (b) can be computed quickly for each element in Step 2.2.

- *Step* 2.1. In this step, $n$ bits are used as approximately $n/\lg n$ counters. Corresponding to positions $pn^{\delta}\lg n$ for $p = 1$ to $n^{1-\delta}/\lg n$, set up and initialize $n^{\delta}$ counters each, namely $C_{p,b}$, for $1 \le p \le n^{1-\delta}/\lg n$, $1 \le b \le n^{\delta}$. There are $n/\lg n$ counters in total. At the end of this step, each counter $C_{p,b}$ will contain the number of (nonguide) elements to the left of, and including, position $p$ that belong to block $b$.

  The values in the counters are obtained in $O(n \lg n)$ comparisons as follows. For each nonguide element $L[i]$,

  perform a binary search to find out the block $b$ to which $L[i]$ belongs (by resolving ties using the $g_i$ counters), and increment the counter $C_{p,b}$ where $p$ is the smallest multiple of $n^{\delta}\lg n$ greater than or equal to $i$.

Now add the successive counters corresponding to each block, so that each counter $C_{p,b}$ contains the required value.

- *Step* 2.2: *Partition all elements stably.* To place a nonguide element $L[i] = x$ in its destination, first find the block $b$ to which it belongs. To find the position of $x$ in block $b$, we read the counter immediately to the left of position $i$ and scan the locations between that position and $i$. Specifically, let $p$ be the largest multiple of $n^\delta \lg n$ smaller than or equal to $i$. There are $C_{p,b}$ elements to the left of position $i$ in the initial array that belong to block $b$ (some of them might already have been moved to block $b$). So skip $C_{p,b}$ positions in block $b$. Count the number $s$ of elements (excluding the guides) between positions $p$ and $i$ that belong to block $b$, and are yet to be moved (i.e., marked 0). Skip $s$ positions marked 0 in block $b$ from the position $C_{p,b} + 1$, and place $x$ in the next position marked 0. Mark that position 1 and continue with the displaced element to chase its cycle, as in the stable permutation sort [13, Section 2].

  The number of comparisons spent in this step is $O(n^\delta \lg n)$ for finding block $b$ and for the scan to compute $s$, for each element. So this step is accomplished using $O(n^{1+\delta} \lg n)$ comparisons and $O(n)$ data moves.

Now all elements are stably partitioned (though unsorted) in their respective blocks. The sort can be completed by sorting each block stably. Phase 3 accomplishes this.

*Phase* 3. Each block now has approximately $n^{1-\delta}$ elements. To sort each block, recurse until the size of the block reduces to $n^\delta$ at which point apply our stable in-place $O(n^2)$ algorithm [13] (with or without using $n$ bits) in each block. As the size of each block is $n^{1-\delta}$, it can be seen that the recursion depth is about $\lg \delta / \lg(1-\delta)$ which is at most $1/\delta$.

Each recursion step takes $O(n^{1+\varepsilon})$ comparisons and $O(n)$ data moves, and the final step takes $O(n^{1+\delta})$ comparisons. Hence, this phase takes $O((1/\delta)n^{1+\varepsilon})$ comparisons and $O((1/\delta)n)$ data moves using $O(1/\delta)$ indices.

So, overall each phase performs $O(n)$ data moves using a total of $2n + O(n^\delta \lg n)$ bits. Interestingly, the bits used by each phase are not required for the next phase, and hence are reused. Stability is maintained, as equal-valued elements retain their original order before and after each phase. Each phase performs $O(n)$ data moves. The dominant step in terms of the number of comparisons is Step 1.1 which requires $O(n^{1+\varepsilon})$ comparisons in the worst case. As the recursion depth in Phase 3 is at most $1/\delta$, the algorithm performs, in total, $O((1/\delta)n^{1+\varepsilon})$ comparisons, $O(n/\delta)$ data moves using $O(1/\delta)$ indices and $2n + o(n)$ bits. As $\varepsilon$ and $\delta$ are fixed positive constants less than 1, the claimed bounds of the theorem follows. □

**3. Encoding Bits.** We now describe a technique to encode bits into data values, or specifically to encode $m$ bits into a portion of the array consisting of $2m$ distinct elements. This technique has proven effective to obtain in-place algorithms for various problems [8], [14], [11], [7]. This technique is used in the next section to eliminate the $2n + o(n)$ bits used by the algorithm of Theorem 1.

We call an *internal buffer* as the portion of the array consisting of $2m$ distinct elements. The $\lfloor m / \lg m \rfloor$ counters (or $m$ bits) are encoded into this internal buffer.

First the buffer is divided into $\lfloor m / \lg m \rfloor$ groups of $2\lceil \lg m \rceil$ elements each. Each group of $2\lceil \lg m \rceil$ elements represents a counter which can store values from 1 to $m$. For $i = 1$

to $\lceil \lg m \rceil$, the $(2i-1)$st and $(2i)$th element form the $i$th bit of the counter. If the $(2i-1)$st element is smaller than the $(2i)$th element, then the $i$th bit of the counter is 0 and it is 1 otherwise. With this encoding, the following observations are immediate, but useful:

1. $\lfloor m/\lg m \rfloor$ counters each capable of storing a value between 1 and $m$ can be encoded in an internal buffer of at most $2m$ distinct elements.
2. Each counter can be read by performing $\lceil \lg m \rceil$ comparisons.
3. Each counter can be restored (to all 0 bits) using $\lceil \lg m \rceil$ comparisons and at most $\lceil \lg m \rceil$ exchanges. Hence the whole buffer can be restored in $O(m)$ time.
4. Two counters can be added using about $2 \lg m$ comparisons and $\lceil \lg m \rceil$ exchanges.
5. Incrementing/decrementing a counter, in the worst case, takes about $\lg m$ exchanges. However, in an amortized sense, $m$ operations of increment followed by $m$ decrements of a counter cost only $2m$ exchanges.
6. Suppose the counters are encoded in an initially sorted array of $2m$ distinct elements. The sorted array can be recovered by making $m$ pairwise comparisons and at most $m$ exchanges.

**4. Stable In-Place $O(n^{1+\varepsilon})$ Sort With $O(n)$ Moves.** The algorithm of Section 2 uses at most $3m$ (actually $2m + o(m)$) bits of extra memory to sort $m$ elements. So, to apply the encoding technique of the last section, we require about $6m$ data elements. Note also that our encoding technique, as stated in the last section, requires only that a pair of elements encoding a bit are not equal. Hence we can encode $m$ bits into $2m$ elements as long as no element occurs more than $m$ times.

The first step in our encoding step is to divide the list stably into three parts of approximately $n/3$ elements each. We then sort each part by applying the algorithm of Section 2, encoding the bits in a portion containing a sufficient number of distinct elements.

Stable partitioning is achieved by first finding the elements called *pivots* $a$, $b$ of ranks $\lfloor n/3 \rfloor$, $\lfloor 2n/3 \rfloor$, respectively, and then by partitioning all elements with respect to them. These pivots can be found using $O(n^{1+\varepsilon})$ comparisons and no data move using our read-only memory selection algorithm [14], [12].

Stable partitioning can be thought of as stably sorting a list of elements with at most five distinct keys, elements that are less than $a$, $L_{<a}$; elements equal to $a$, $L_{=a}$; elements that are between $a$ and $b$, $L_{a<x<b}$; elements equal to $b$, $L_{=b}$; and elements greater than $b$, $L_{>b}$. Figure 2 depicts an array after stable partitioning. This partitioning can be done using $O(n \lg^* n)$ comparisons and $O(n)$ data moves using the stable in-place constant-key sort [13]. (Recall that $\lg^*$ denotes the number of times the logarithm may be taken before the quantity becomes at most zero.)

Once stable partitioning is done, we sort each part by encoding the required bits into a portion of the list consisting of sufficient distinct keys. Here we outline the details of

| $L_{<a}$ | $L_{=a}$ | $L_{a<x<b}$ | $L_{=b}$ | $L_{>b}$ |
|---|---|---|---|---|

**Fig. 2.** Array after partitioning.

such encoding. To apply the algorithm of Section 2 to a block of size $m$, we require at most $3m$ bits or $3m$ pairs of distinct elements.

*Case* 1: $a = b$. In this case $L_{a<x<b} = \emptyset$ (the empty set) and $L_{=a} = L_{=b} > \lfloor n/3 \rfloor$. Only the sets $L_{<a}$ and $L_{>b}$ need to be sorted. Without loss of generality let $|L_{<a}| \leq |L_{>b}|$ (the other case is symmetric). We first sort $L_{<a}$ as follows. Stably partition $L_{<a}$ into five parts after finding the two elements of appropriate (equally spaced) ranks as before. Let $x = |L_{<a}|/3$. Recall that $3x = |L_{<a}| \leq |L_{>b}| < |L_{=a}|$. Hence, $3x$ bits can be encoded into $3x$ distinct pairs, one element for the pair coming from $L_{>b}$ and the other from $L_{=a}$. Thus each block of $L_{<a}$ and hence the entire $L_{<a}$ is sorted by encoding the bits in $L_{=a}$ and $L_{>b}$.

After the set $L_{<a}$ is sorted, recurse on the set $L_{>b}$ to sort $L_{>b}$. As recursion is applied on the tail, no stack space is required to remember the indices. Also, as the recursion occurs on a portion of size less than $n/3$, the time complexity of the algorithm is dominated by the other steps.

*Case* 2: $a \neq b$. Here, only $L_{<a}$, $L_{a<x<b}$ and $L_{>b}$ are to be sorted, and no two of the five blocks have any element in common. So, bits can be encoded in any of the pairs of blocks.

First sort the set $L_{a<x<b}$, as in Case 1, as follows. Stably partition the set into five blocks by first finding the two appropriately (equally spaced) ranked elements, and sort each block by encoding the bits into a sufficient number of elements in $L_{<a} \cup L_{=a}$ and $L_{=b} \cup L_{>b}$. There is a sufficient number of distinct pairs to sort each block since the size of each block of $L_{a<x<b}$ is at most $n/9$ and $|L_{<a} \cup L_{=a}| \geq n/3$ and $|L_{=b} \cup L_{>b}| \geq n/3$.

Without loss of generality, let $|L_{<a}| \leq |L_{>b}|$. Sort $L_{<a}$, as before, by encoding the bits into the set $L_{>b}$ and the $|L_{>b}|$ elements preceding the set $L_{>b}$. As $a \neq b$, there is a sufficient number of distinct elements to encode $|L_{<a}|$ bits (as $|L_{<a}| \leq |L_{>b}|$) in those two blocks. Thus $L_{<a}$ can be stably partitioned first into five parts, and each block of $L_{<a}$ can be stably sorted by encoding the bits into $L_{>b}$ and a sufficient number of elements preceding them.

After the set $L_{<a}$ is sorted, recurse on the set $L_{>b}$ to sort $L_{>b}$. As recursion is applied on the tail, no stack space is required to remember the indices. Also, as the recursion occurs on a portion of size less than $n/3$, the time complexity of the algorithm is dominated by the other steps.

4.1. *Analysis.* Here we give the complete analysis of the algorithm with the effect of bit encoding.

In Step 1.1, first $n$ bits are initialized to 0 which require $n$ exchanges. Then $n^\delta$ bits are set which requires $n^\delta$ data moves once these bits are encoded. Also, for the $n^\delta$ counters $g_i$'s, just writing down the values requires $n^\delta \lg n$ data moves. In Step 1.2 the tags are reset $O(n)$ times requiring $O(n)$ data moves.

In Step 2.1 the $n/\lg n$ counters can be initialized in $O(n)$ time due to observation 3 of Section 3. Then one counter is incremented for every element, thus using $O(n)$ data moves as a consequence of observation 5. Determining the final block of each (nonguide) element may involve reading $n^\delta$ counters for each binary search step, thus requiring a total of $O(n^{1+\delta} \lg^2 n)$ comparisons. Then there are about $2n/\lg n$ counter additions (each

counter is involved in at most two additions) which involves a total of $O(n)$ comparisons and data moves due to observation 4.

In Step 2.2, first $n$ bits are initialized to 0 which requires $n$ exchanges. Then the final block of each element is found, which involves $O(n^{\delta} \lg^2 n)$ comparisons for each element. Then the counter $C_{p,b}$ is read for the element which involves $O(\lg n)$ comparisons per element.

The scan to compute $s$ may require reading the $n^{\delta}$ $(g_i)$ counters, and hence $n^{\delta} \lg n$ comparisons, for each of the $n^{\delta} \lg n$ possible elements to determine whether or not it belongs to block $b$. Thus, for each element, $O(n^{2\delta} \lg^2 n)$ comparisons are spent in this step. Thus this step requires $O(n^{1+2\delta} \lg^2 n)$ comparisons and $O(n)$ data moves in the worst case.

After sorting each block, the internal buffer can be restored in $O(n)$ time due to observation 3. Also a sorted list in which the bits are encoded can be recovered in $O(n)$ time due to observation 6. Furthermore, each partition (in Cases 1 and 2) is partitioned again into only a constant number of parts to complete the sort of that partition. Also, the recursion in Case 1 occurs on only one set of size less than $n/3$ and hence the time spent will be dominated by the time spent in the step which sorts each block by encoding counters in the others. Repeating the entire sort in each partition results in an increase only in the constant factor.

Thus the bit encoding results in an extra $O(n^{1+2\delta} \lg^2 n)$ comparisons (Step 2.2) and $O(n)$ data moves. As $\delta < \varepsilon/2$, the algorithm still performs $O(n^{1+\varepsilon})$ comparisons. Thus we have the following theorem.

THEOREM 2. *A list of n elements can be stably sorted in-place using $O(n)$ data moves and $O(n^{1+\varepsilon})$ comparisons for any fixed positive constant $\varepsilon < 1$.*

**5. Conclusions.** We have developed an $O(n^{1+\varepsilon})$ stable in-place sort that performs $O(n)$ data moves, where $\varepsilon$ is an arbitrary, but fixed, constant. The main bottleneck (for $\varepsilon$) of the algorithm is the $O(n^{1+\varepsilon})$ selection algorithm of [12]. Hence, as is the case for the selection algorithm, $\varepsilon$ can be parametrized to a function of $n$ in the range $((\lg \lg n/3 \lg n)^{1/2}, 1)$. Furthermore, a decrease in the value of $\varepsilon$ results in an exponential (in $1/\varepsilon$) factor increase in the runtime of the algorithm, and a linear (in $1/\varepsilon$) factor increase in the space requirement and the number of moves performed.

Our algorithm improves the previous upper bound of $O(n^2)$ for the problem and establishes that the upper bound for the stable version is the same as the best known upper bound for the unstable version for sorting in-place with $O(n)$ data moves. The result and the algorithm we develop show further evidence to the proposition that though stability is a difficult constraint to achieve in-place, asymptotically it does not cost more than a constant factor.

En route to our algorithm, we came across several techniques to design space efficient algorithms. These include:

- Bit encoding (which has been used recently in a variety of problems [11], [7], [8], [13] to obtain space-efficient algorithms).
- The concept of computing and storing information for a sparse subset of positions

as in Step 2.2.1 (which is used to obtain a fast stable in-place algorithm to sort a list consisting of two distinct keys [13]).

Further applications of these space-saving techniques may be useful to obtain good upper bounds on time–space tradeoffs for similar problems.

The main open problem is to design an in-place $O(n \lg n)$ worst-case (stable or unstable) algorithm that performs $O(n)$ data moves or to prove that it is not possible. This problem has been settled for the average case for the unstable version [11]. It is not even clear whether having $o(n)$ extra storage cells helps achieve the $O(n \lg n)$ bound for stable sorting with $O(n)$ data moves. Again, this was achieved for the unstable case in [11].

## References


[1] E. H. Friend, Sorting on Electronic Computers, *Journal of the ACM*, **3**(2) (1956), 134–168.

[2] B. C. Huang and M. A. Langston, Practical In-place Merging, *Communications of the ACM*, **31**(3) (1988), 348–352.

[3] B. C. Huang and M. A. Langston, Fast Stable Merging and Sorting in Constant Extra Space, *Proceedings of the International Conference on Computing and Information*, 1989, pp. 71–80.

[4] J. Katajainen and T. Pasanen, Sorting Multisets Stably in Minimum Space, *Proceedings of the 3rd Scandinavian Workshop on Algorithm Theory*, Lecture Notes in Computer Science, Vol. 621, Springer-Verlag, Berlin, 1992, pp. 410–421.

[5] D. E. Knuth, *The Art of Computer Programming*, Vol. III, Addison-Wesley, Reading, MA, 1973.

[6] M. A. Kronrod, Optimal Ordering Algorithm Without Operational Field, *Soviet Math. Dokl.*, **10** (1969), 744–746.

[7] T. W. Lai and D. Wood, Implicit Selection, *Proceedings of the 1st Scandinavian Workshop on Algorithm Theory*, Lecture Notes in Computer Science, Vol. 318, Springer-Verlag, Berlin, 1988, pp. 14–23.

[8] J. I. Munro, An Implicit Data Structure Supporting Insertion, Deletion, and Search in $O(\lg^2 n)$ Time, *Journal of Computer and System Sciences*, **21** (1980), 236–250.

[9] J. I. Munro and V. Raman, Fast Stable In-Place Sorting with $O(n)$ Data Moves, *Proceedings of the 11th FST & TCS Conference*, New Delhi, Lecture Notes in Computer Science, Vol. 560, Springer-Verlag, Berlin, 1991, pp. 266–277.

[10] J. I. Munro and V. Raman, Sorting Multisets and Vectors In-Place, *Proceedings of the 2nd Workshop on Algorithms and Data Structures*, Lecture Notes in Computer Science, Vol. 519, Springer-Verlag, Berlin, 1991, pp. 473–480.

[11] J. I. Munro and V. Raman, Sorting with Minimum Data Movement, *Journal of Algorithms*, **13** (1992), 374–393.

[12] J. I. Munro and V. Raman, Selection from Read-Only Memory and Sorting with Optimum Data Movement, *Proceedings of the 12th FST & TCS Conference*, New Delhi, Lecture Notes in Computer Science, Vol. 652, Springer-Verlag, Berlin, 1992, pp. 380–391; to appear in *Theoretical Computer Science*.

[13] J. I. Munro, V. Raman, and J. S. Salowe, Stable In Situ Sorting and Minimum Data Movement, *BIT*, **30** (1990), 220–234.

[14] V. Raman, Sorting In-Place with Minimum Data Movement, Ph.D. Thesis, Technical Report CS 91-12, Department of Computer Science, University of Waterloo (1991).

[15] L. Trabb Pardo, Stable Sorting and Merging with Optimal Space and Time Bounds, *SIAM Journal of Computing*, **6** (1977), 351–372.

[16] J. W. J. Williams, Algorithm 232, Heapsort, *Communications of the ACM*, **7** (1964), 347–348.